

Reporte Técnico de Actividades Práctico- Experimentales Nro. 003

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	• Jose Camilo Merino Morocho • Edgar Fabricio Rios Cuenca • Ashley Patricia Mocha Valarezo • Abraham Daniel Maza Lapo
Asignatura	Matemáticas Discretas
Ciclo	1 A
Unidad	3
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Tipo	Grupal
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	29 de junio al 24 de julio de 2026
Horario	Lunes 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	12 horas (2 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).
- Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.



- Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.

3. Materiales, Reactivos, Pizarra y marcadores de colores

- Proyector multimedia
- Hojas de trabajo con ejercicios de teoría de grafos y árboles
- Guía paso a paso de los algoritmos BFS, DFS y Dijkstra
- Plantillas para construcción de grafos y árboles
- Investigación sobre aplicaciones de grafos en redes y optimización

4. EQUIPOS Y HERRAMIENTAS

- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL
- Laboratorio de computación
- Software Graph Online (<https://graphonline.ru/es>) o Gephi para modelado de grafos
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026)

1. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
2. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).



FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
2. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

FASE 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 julio 2026)

1. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.
2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un mini-informe con exposición apoyada en recursos TIC.

FASE 4: Proyecto Integrador - Modelado y Optimización de Redes (Semana 16: 20-24 julio 2026)

1. Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)

FASE 5: Exposición, Defensa y Evaluación Final (Semana 17: 27-31 julio 2026)

ATENCIÓN: Esta semana está destinada a actividades de fin de ciclo (29-31 julio: socialización de proyectos de investigación, casas abiertas, ferias científicas). Las actividades APE se realizan al inicio de la semana (27-28 julio).

1. Exposición y defensa oral del proyecto integrador ante la clase: cada equipo presenta su trabajo (10-15 minutos), explicando el problema abordado, la metodología aplicada (lógica + booleana + grafos), los resultados obtenidos y las conclusiones.

5. Resultados

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026)

1. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

CONTAGIOS DE UNA PANDEMIA EN UN GRUPO DE PERSONAS

Durante una pandemia para poder comprender mejor cómo se propaga una enfermedad, se utilizará un grafo, donde cada persona será representada por un vértice y cada contacto entre dos personas será representado por una arista. De esta forma, se podrá analizar la red de contagios e identificar las personas con mayor riesgo de transmitir la enfermedad. Supongamos que en un grupo de **7 personas** una de ellas estaba infectada y, después de varios días, se registraron los siguientes contactos:

- Ana tuvo contacto con Bruno y Carlos.
- Bruno tuvo contacto con Ana, Diana y Elena.
- Carlos tuvo contacto con Ana y Fernando.
- Diana tuvo contacto con Bruno y Gabriela.
- Elena tuvo contacto con Bruno y Gabriela.
- Fernando tuvo contacto con Carlos y Gabriela.
- Gabriela tuvo contacto con Diana, Elena y Fernando.

CANTIDAD DE VERTICES	
Vértice	Personas
A	Ana
B	Bruno
C	Carlos
D	Diana
E	Elena
F	Fernando
G	Gabriela

Hay 7 vértices porque existen 7 personas:

$V = \{A, B, C, D, E, F, G\}$

CANTIDAD DE ARISTAS	
Arista	Significado
A – B	Ana tuvo contacto con Bruno
A – C	Ana tuvo contacto con Carlos
B – D	Bruno tuvo contacto con Diana
B – E	Bruno tuvo contacto con Elena
C – F	Carlos tuvo contacto con Fernando



D – G	Diana tuvo contacto con Gabriela
E – G	Elena tuvo contacto con Gabriela
F – G	Fernando tuvo contacto con Gabriela

Hay 8 aristas, ya que existen ocho contactos entre las personas.

$E = \{(A,B), (A,C), (B,D), (B,E), (C,F), (D,G), (E,G), (F,G)\}$

CANTIDAD DE GRADOS		
Persona	Contactos	Grado
Ana	Bruno, Carlos	2
Bruno	Ana, Diana, Elena	3
Carlos	Ana, Fernando	2
Diana	Bruno, Gabriela	2
Elena	Bruno, Gabriela	2
Fernando	Carlos, Gabriela	2
Gabriela	Diana, Elena, Fernando	3

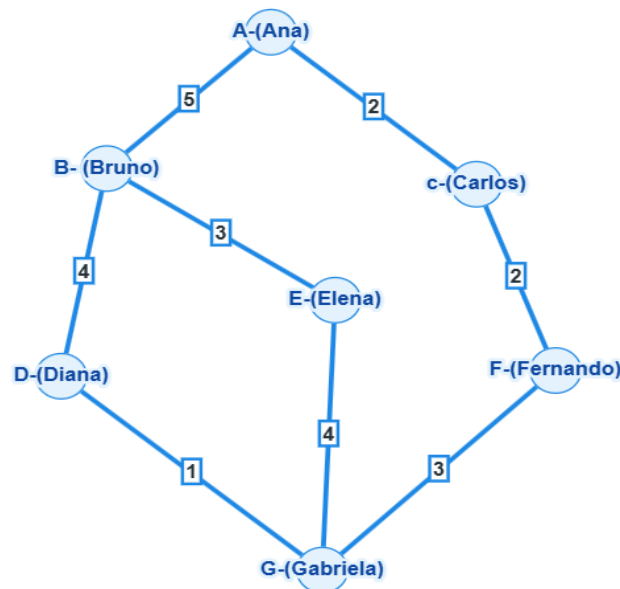
El grado indica cuántos contactos tuvo cada persona.

PESO	
Arista	Peso
Ana – Bruno	5
Ana – Carlos	2
Bruno – Diana	4
Bruno – Elena	3
Carlos – Fernando	2
Diana – Gabriela	1
Elena – Gabriela	4
Fernando – Gabriela	3

Aquí detallamos el numero de veces que se encuentran entre cada persona del grupo de amigos.

Propiedades estructurales

- **Tipo de grafo:** No dirigido, porque el contacto entre dos personas es mutuo.
 - **Grafo conectado:** Sí, todas las personas están relacionadas mediante uno o varios contactos.
 - **Vértices con mayor grado:** Bruno y Gabriela, ya que tuvieron **3 contactos** cada uno.
 - **Posibles focos de propagación:** Bruno y Gabriela podrían ser las personas con mayor probabilidad de transmitir la enfermedad debido a que estuvieron en contacto con más personas.
2. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando contagios de una pandemia en un grupo de personas, visualizándolo con el software y analizando sus propiedades (conectividad, caminos, mínimos, nodos críticos)



Análisis de Propiedades:

Propiedad	Descripción	Resultado en el grafo
Conectividad	Indica si todos los vértices están conectados mediante uno o varios caminos.	El grafo es conectado , ya que desde cualquier persona es posible llegar a cualquier otra siguiendo las aristas. No existen vértices aislados ni componentes separadas.
Camino mínimo	Es la ruta con el menor costo total, calculado como la suma de los pesos de las aristas.	De Ana (A) a Gabriela (G) existen tres rutas: $A \rightarrow B \rightarrow D \rightarrow G = 10$, $A \rightarrow B \rightarrow E \rightarrow G = 12$ y $A \rightarrow C \rightarrow F \rightarrow G = 7$. El camino mínimo es $A \rightarrow C \rightarrow F \rightarrow G$, con un costo total de 7.

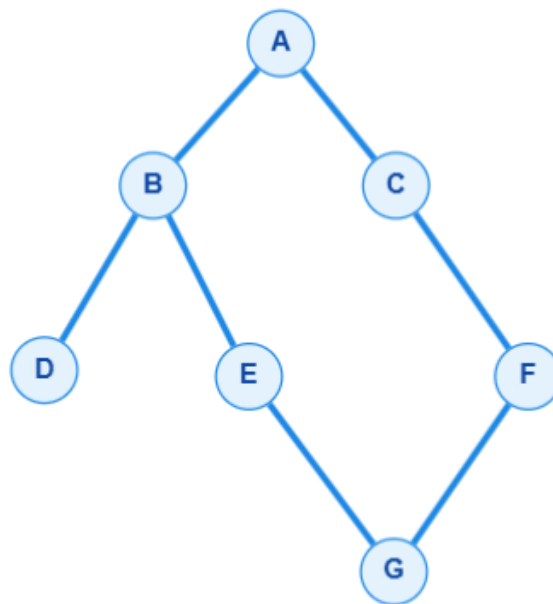
Nodos críticos	Son los vértices cuya eliminación afecta la comunicación o la propagación dentro de la red.	Bruno (B) y Gabriela (G) son los nodos críticos, ya que poseen el mayor número de conexiones (grado 3) y enlazan diferentes partes del grafo. Si uno de ellos se elimina o se aísla, la red pierde conectividad y la propagación cambia significativamente.
-----------------------	---	--

FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.

Taller colaborativo de aplicación de algoritmos

Se utilizó el siguiente grafo para aplicar manualmente los algoritmos BFS, DFS y Dijkstra.



Algoritmo BFS (Breadth First Search)

El algoritmo BFS o búsqueda en anchura recorre un grafo visitando primero todos los nodos cercanos al nodo inicial antes de continuar con los siguientes niveles. Para ello utiliza una cola (FIFO).

- **Inicio:** Se comienza en el nodo origen **A**, el cual se marca como visitado/descubierto y se ingresa en la cola. Inmediatamente se extrae y se agregan sus vecinos contiguos directos **B** y **C** a la cola.
- **Paso 1:** Se procesa el siguiente elemento al frente de la cola, el nodo **B**, marcándolo como visitado. Se inspeccionan sus vecinos no descubiertos y se añaden al final de la cola los nodos **D** y **E**.
- **Paso 2:** Se extrae de la cola el nodo **C**. Se explora su adyacencia y se incorpora el nodo **F** a la estructura.

- **Paso 3:** Se continúa el vaciado secuencial de la cola procesando consecutivamente los nodos **D** y **E**. Al evaluar los vecinos de **E**, se descubre el nodo **G** y se encola.
- **Paso 4:** Finalmente, se extraen y procesan los nodos restantes **G** y **F**. Como todos los nodos del grafo han sido completamente descubiertos y validados, la cola queda vacía y el algoritmo concluye.

Resultado: Orden de descubrimiento

A → B → C → D → E → G → F

BFS permitió recorrer el grafo por niveles, visitando primero todos los vecinos del nodo inicial antes de continuar con los demás.

Algoritmo DFS (Depth First Search)

El algoritmo DFS (Depth First Search) o búsqueda en profundidad explora un camino hasta donde sea posible antes de regresar para continuar con otra rama. Utiliza una pila (LIFO).

- **Inicio:** Se define el punto de partida en el nodo **A**, colocándolo en el tope de la pila de exploración.
- **Paso 1:** Desde el nodo **A** se avanza profundamente hacia su primer vecino disponible **B**. Siguiendo la política de máxima profundidad, desde **B** se progresa de inmediato hacia **D**, alcanzando un nodo hoja (terminal) dentro de esta ramificación.
- **Paso 2:** Al no existir más vecinos sin visitar desde **D**, se ejecuta el mecanismo de retroceso o *backtracking* regresando la ejecución hasta el nodo anterior activo (**B**). Desde ahí se redirige la exploración hacia la siguiente ruta inexplorada: el nodo **E**.
- **Paso 3:** Desde el vértice **E** se continúa el avance lineal hacia el nodo **G**. De forma sucesiva, desde **G** se explora hacia **F**, y finalmente desde **F** se alcanza el nodo **C**, completando de forma exhaustiva la totalidad de los componentes del grafo.

Resultado: Orden de descubrimiento

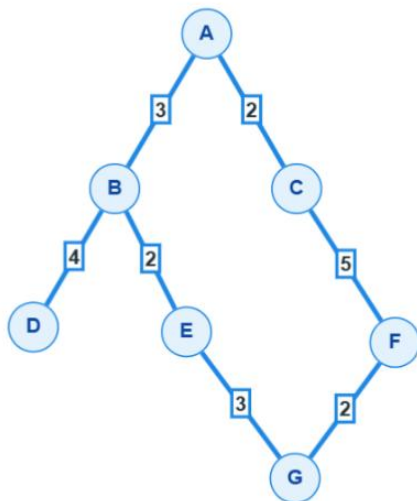
A → B → D → E → G → F → C

DFS exploró completamente cada rama antes de regresar, permitiendo recorrer el grafo en profundidad.

Algoritmo de Dijkstra (Camino más corto)

El algoritmo de Dijkstra permite encontrar el camino de menor costo entre un nodo origen y un nodo destino en un grafo ponderado. Para ello, selecciona en cada iteración el nodo con la menor distancia acumulada y actualiza las distancias de sus nodos vecinos hasta obtener la ruta más corta.

Grafo ponderado



Arista	Peso
A – B	3
A – C	2
B – D	4
B – E	2
C – F	5
E – G	3
F – G	2

Nodo origen: A

Nodo destino: G

Desarrollo

- **Inicial:** Se inicializan las etiquetas de distancia. La distancia acumulada del nodo origen **A** es 0, mientras que la distancia provisional de todos los demás nodos restantes se establece en infinito (∞). El conjunto de nodos evaluados y definitivos se encuentra vacío.
- **Iteración 1:** Se selecciona el nodo con la distancia mínima actual (**A**, costo 0) y se marca como confirmado/permanente. Se examinan sus aristas adyacentes y se realiza la relajación: la distancia hacia **B** pasa de ∞ a 3 ($0 + 3$), y la distancia hacia **C** disminuye a 2 ($0 + 2$).
- **Iteración 2:** El nodo no confirmado con la menor distancia acumulada es **C** (costo 2). Se fija como permanente. Evaluando su periferia, se calcula la distancia acumulada proyectada hacia su vecino **F**, la cual se actualiza con un valor de 7 ($2 + 5$).
- **Iteración 3:** El nodo menor restante es **B** (costo 3). Se establece como permanente. Se inspeccionan sus vecinos directos para actualizar métricas: la distancia hacia **D** pasa a ser 7 ($3 + 4$) y la distancia óptima temporal hacia el nodo **E** se reduce a 5 ($3 + 2$).
- **Iteración 4:** Se selecciona y confirma el nodo **E** (costo 5) por poseer el menor valor provisional. Desde **E**, se evalúa el camino hacia su único vecino pendiente **G**, computando y asignando una distancia acumulada de 8 ($5 + 3$).
- **Iteración 5:** Se evalúan y fijan los nodos restantes **D** (costo 7) y **F** (costo 7). Al inspeccionar las conexiones existentes (por ejemplo, de **F** hacia **G** el costo sería $7 + 2 = 9$), se comprueba que ninguna de estas rutas alternativas mejora o reduce la distancia ya calculada previamente hacia **G**.
- **Iteración 6:** Se selecciona formalmente el nodo de destino **G** (costo 8), consolidando su estado como nodo definitivo y dando por finalizadas las operaciones del algoritmo.

Resultado

Camino mínimo: A $\rightarrow$ B $\rightarrow$ E $\rightarrow$ G

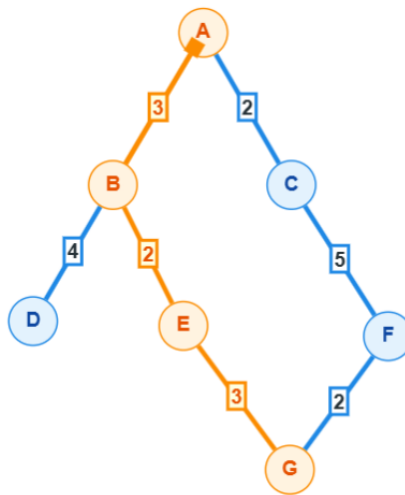
Costo total: 8

- Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

Modelado de la red mediante software

Se ingresó el mismo grafo en **Graph Online**, asignando los mismos pesos a cada arista y ejecutando el algoritmo de Dijkstra con **A** como nodo origen y **G** como nodo destino.

La longitud del camino más corto es 8: A⇒B⇒E⇒G



Resultado obtenido

El software encontró el siguiente camino mínimo:

A → B → E → G Costo total: 8

Comparación

Característica	Manual	Software
Nodo origen	A	A
Nodo destino	G	G
Camino mínimo	A → B → E → G	A → B → E → G
Costo total	8	8

Los resultados obtenidos manualmente coinciden con los generados por **Graph Online**, comprobando que el algoritmo de Dijkstra fue aplicado correctamente. El cálculo manual permitió comprender el procedimiento paso a paso, mientras que el software facilitó la verificación del camino mínimo y del costo total.

9. Reflexión crítica

EN PROCESO



11. Bibliografía

- O. Levin, *Discrete Mathematics: An Open Introduction*, 4.a ed. Greeley, CO, EE. UU.: Oscar Levin, 2025.
- J. F. Villalpando Becerra y A. García Sandoval, *Matemáticas discretas: aplicaciones y ejercicios*. México D.F., México: Grupo Editorial Patria, 2014.
- S. S. Epp, *Matemáticas discretas con aplicaciones*, 4.a ed. México D.F., México: Cengage Learning, 2012.